

WiDS Program Report: Learnings and Experiments

Yash Meshiya

25B1046

Contents

1	Introduction	3
2	Overview of the WiDS Program	3
3	Foundations of Reinforcement Learning	3
4	Multi-Armed Bandit Problems	3
4.1	Algorithms Studied	4
4.2	Experimental Observations	4
5	Markov Decision Processes	4
5.1	MDP Formulation	4
5.2	Key Insights	4
6	Dynamic Programming Methods	4
6.1	Algorithms Implemented	4
6.2	Learning Outcomes	5
7	Monte Carlo Methods	5
7.1	Experiments	5
7.2	Observations	5
8	Temporal Difference Learning	5
8.1	Algorithms Studied	5
8.2	Results	5
9	Function Approximation	5
9.1	Approaches Used	6
9.2	Challenges	6
10	Challenges and Debugging	6
11	Key Learnings and Reflections	6
12	Conclusion	6

1. Introduction

The Women in Data Science (WiDS) program provided a structured and rigorous introduction to data science and machine learning, with a strong emphasis on reinforcement learning (RL). This report documents my learning journey throughout the program, combining theoretical understanding with practical experimentation. The objective of this report is to summarize the key concepts learned, experiments conducted, challenges encountered, and insights gained during the course of WiDS.

2. Overview of the WiDS Program

The WiDS curriculum was designed to balance theory and practice. It covered core data science concepts, probability, optimization, and machine learning, gradually progressing toward reinforcement learning. Emphasis was placed on understanding mathematical foundations, implementing algorithms from scratch, and analyzing their behavior through experiments and visualizations.

3. Foundations of Reinforcement Learning

Reinforcement learning is a paradigm of machine learning in which an agent learns by interacting with an environment. The agent observes the current state, takes an action, receives a reward, and transitions to a new state. Over time, the agent aims to learn a policy that maximizes cumulative reward.

Key components of reinforcement learning studied during WiDS include:

- Agent and environment interaction
- States, actions, and rewards
- Policies (deterministic and stochastic)
- Return and discount factor
- Exploration versus exploitation trade-off

Understanding these components was essential before moving on to algorithmic implementations.

4. Multi-Armed Bandit Problems

The multi-armed bandit problem was the first formal reinforcement learning setting explored. It models decision-making under uncertainty where actions have unknown reward distributions.

4.1 Algorithms Studied

The following algorithms were implemented and analyzed:

- Epsilon-greedy action selection
- Optimistic initial value method
- Upper Confidence Bound (UCB)

4.2 Experimental Observations

Experiments showed that purely greedy strategies perform poorly due to lack of exploration. Epsilon-greedy methods balance exploration and exploitation, while UCB methods provide a more principled approach by considering uncertainty in value estimates. These experiments highlighted the importance of controlled exploration in learning optimal behavior.

5. Markov Decision Processes

Markov Decision Processes (MDPs) provide a mathematical framework for modeling sequential decision-making problems. An MDP is defined by a set of states, actions, transition probabilities, rewards, and a discount factor.

5.1 MDP Formulation

I practiced formulating environments such as the Frozen Lake problem and custom guessing games as MDPs. This required careful definition of terminal states, reward structures, and transition dynamics.

5.2 Key Insights

Correct MDP formulation significantly impacts learning performance. Even small errors in reward design or transition modeling can lead to unstable or misleading results.

6. Dynamic Programming Methods

Dynamic programming (DP) methods assume full knowledge of the environment model.

6.1 Algorithms Implemented

- Policy Evaluation
- Policy Iteration

- Value Iteration

6.2 Learning Outcomes

DP methods clearly demonstrated the Bellman equations and the principle of optimality. However, their reliance on a known model limits their applicability in real-world problems.

7. Monte Carlo Methods

Monte Carlo (MC) methods learn value functions using complete episodes without requiring a model of the environment.

7.1 Experiments

Both first-visit and every-visit Monte Carlo prediction methods were implemented. MC control using epsilon-greedy policies was also explored.

7.2 Observations

MC methods provide unbiased value estimates but suffer from high variance. Their episodic nature makes them unsuitable for continuing tasks without modification.

8. Temporal Difference Learning

Temporal Difference (TD) learning combines ideas from dynamic programming and Monte Carlo methods.

8.1 Algorithms Studied

- TD(0)
- SARSA (on-policy control)
- Q-learning (off-policy control)

8.2 Results

TD methods converged faster than Monte Carlo methods and were more sample-efficient. Q-learning, in particular, demonstrated robustness due to its off-policy nature.

9. Function Approximation

To handle large or continuous state spaces, function approximation techniques were introduced.

9.1 Approaches Used

- Linear function approximation
- Neural network-based value approximation

9.2 Challenges

While function approximation enables generalization, it can introduce instability and divergence if not carefully designed. This highlighted the importance of learning rates and feature selection.

10. Challenges and Debugging

Throughout the WiDS program, several challenges were encountered:

- Incorrect MDP definitions
- Indexing and state-mapping bugs
- Non-converging learning algorithms

Debugging techniques such as visualization of value functions, reward tracking, and step-by-step simulation proved highly effective.

11. Key Learnings and Reflections

The most important learning was the strong connection between theory and implementation. Reinforcement learning algorithms are highly sensitive to design choices, and empirical validation is essential. The program also improved my ability to read research-oriented material and translate it into working code.

12. Conclusion

The WiDS program provided a strong foundation in data science and reinforcement learning. By combining mathematical rigor with hands-on experimentation, it prepared me for advanced coursework, research, and real-world applications. The concepts learned during this program will serve as a cornerstone for future work in machine learning and artificial intelligence.